



**UNIVERSIDADE DE PERNAMBUCO
LICENCIATURA EM COMPUTAÇÃO**

**ANDRISSON HEITOR SOUSA DE MELO
JAIR VINICIOS DA SILVA CALADO
NICOLAS EDWARD ALVES DE ALMEIDA
PAULO RICARDO DA SILVA FERREIRA**

**PROJETO DE BANCO DE DADOS
GARANHUNS 2026**

SUMÁRIO

1. INTRODUÇÃO	03
2. FERRAMENTAS	03
3. MODELO CONCEITUAL	03
3.1. MINI MUNDO	03
3.2. TODAS AS TABELAS	04
3.3. PRINCIPAIS RELACIONAMENTOS	04
3.4. MODELO ENTIDADE-RELACIONAMENTO	06
4. MODELO LÓGICO	07
4.1. DIAGRAMA ENTIDADE-RELACIONAMENTO	07
5. MODELO FÍSICO	12
5.1. POVOAMENTO	12
6. PRINCIPAIS VIEWS	18
7. PRINCIPAIS FUNÇÕES	21
8. PRINCIPAIS TRIGGERS	25
9. CONSIDERAÇÕES FINAIS	27

GestLab TI

Sistema de Agendamento de Laboratório de Informática

Trabalho apresentado à Universidade de Pernambuco (UPE) como requisito para conclusão da disciplina de Banco de Dados do curso de Licenciatura em Computação.

1. Introdução

Os bancos de dados são uma parte fundamental da tecnologia moderna. Eles são usados para armazenar, gerenciar e recuperar dados de uma ampla variedade de aplicações, desde sistemas de comércio eletrônico a sistemas de saúde.

Na atualidade, os bancos são ainda mais importantes do que nunca. Com o aumento da quantidade de dados gerados todos os dias, os bancos de dados são essenciais para garantir que esses dados sejam armazenados de forma segura e eficiente.

Este trabalho para a disciplina de banco de dados tem como objetivo desenvolver um projeto completo implementado em SQL de um **Sistema de Agendamento e Gerenciamento de Laboratórios de Informática**. O objetivo principal do sistema é melhorar a organização e otimizar o controle dos recursos e infraestrutura disponíveis.

2. Ferramentas

Para desenvolver o MER (Modelo Entidade Relacionamento), foi utilizada a ferramenta ERDPlus. Para a criação de tabelas e modelagem do banco de dados usando a linguagem SQL, foi utilizada a ferramenta Supabase, e para a aplicação do banco de dados foi utilizada a ferramenta VScode.

3. Modelo Conceitual

3.1. Mini Mundo

A reitoria e a coordenação de tecnologia de uma Universidade contrataram um desenvolvedor de software para criar uma solução integrada para o acompanhamento, reserva e manutenção de seus laboratórios de informática. O analista de sistemas realizou reuniões de definição de escopo com os principais envolvidos (Coordenação de Cursos, Equipe de TI e Corpo Docente) para o levantamento inicial de requisitos, identificando a necessidade de centralizar a gestão de infraestrutura acadêmica, otimizar o fluxo de reservas e garantir a agilidade no suporte técnico.

As reuniões identificaram que o sistema deverá conter um controle rígido de usuários, cadastro de infraestrutura física (laboratórios) e um catálogo dos ativos disponíveis (computadores, softwares e tipos de equipamentos). Para a operação diária, mapeou-se a necessidade de telas para solicitação e acompanhamento de reservas associadas a horários específicos, cursos e disciplinas. No âmbito do suporte, o sistema deve permitir que os

usuários relatem incidentes por meio de categorias de ocorrências, gerando ordens de serviço (manutenção) que serão atribuídas e executadas pela equipe de técnicos.

04

3.2. Todas as Tabelas

Lista de Tabelas do Banco de Dados

Sistema de Gestão de Laboratórios de Informática

Tabela	Tipo	Descrição
USUARIO	Entidade Base	Dados de autenticação de todos os usuários
ALUNO	Especialização	Herda de USUARIO
PROFESSOR	Especialização	Herda de USUARIO
TECNICO	Especialização	Herda de USUARIO
MONITOR	Especialização	Herda de USUARIO
CURSO	Apoio	Cursos da instituição
DISCIPLINA	Apoio	Matérias ofertadas
PROFESSOR_DISCIPLINA	Junção (N:M)	Relação Professor × Disciplina
LABORATORIO	Principal	Laboratórios de informática
RESERVA	Principal	Agendamentos de laboratório
HORARIO	Apoio	Catálogo de horários
RESERVA_HORARIO	Junção (N:M)	Horários de cada reserva
MONITOR_LABORATORIO	Junção (N:M)	Alocação de monitores
OCORRENCIA	Principal	Registro de problemas
CATEGORIA_OCORRENCIA	Apoio	Tipos de ocorrência
MANUTENCAO	Principal	Manutenções realizadas
EQUIPAMENTO	Principal	Equipamentos dos laboratórios
COMPUTADOR	Especialização	Herda de EQUIPAMENTO
TIPO_EQUIPAMENTO	Apoio	Classificação de equipamentos
SOFTWARE	Principal	Softwares disponíveis
COMPUTADOR_SOFTWARE	Junção (N:M)	Softwares instalados nos PCs
PROFESSOR_ESPECIALIDADE	Junção	Especialidades dos professores
TECNICO_ESPECIALIDADE	Junção	Especialidades dos técnicos

3.3. Principais Relacionamentos do Banco de Dados

Herança de Usuário

- A tabela **USUÁRIO** funciona como entidade base (superclasse);
- As tabelas **ALUNO**, **PROFESSOR**, **TÉCNICO** e **MONITOR** são especializações (herdam os atributos de **USUÁRIO**);
- Essa estrutura permite que qualquer tipo de usuário (aluno, professor, técnico ou monitor) possa interagir com o sistema de forma padronizada.

Herança de Equipamento

- A tabela **EQUIPAMENTO** é a entidade base para todos os itens físicos do laboratório;
- A tabela **COMPUTADOR** é uma especialização de **EQUIPAMENTO**, herdando seus atributos e adicionando campos específicos (como `numero_maquina`, e `sistema operacional`);
- Essa modelagem permite expandir o sistema futuramente para outros tipos de equipamentos (ex: projetor, switch, etc.).

Fluxo de Agendamento (Reserva)

- A tabela **RESERVA** é o centro do processo de agendamento;
- Ela se relaciona com:
 - **PROFESSOR_DISCIPLINA** (para garantir que o professor só reserve para disciplinas que leciona)
 - **LABORATORIO** (local da reserva)
 - **HORARIO** (através da tabela associativa **RESERVA_HORARIO**)
- Relacionamento **N:M** entre Reserva e Horário.

Gestão de Ocorrências

- A tabela **OCORRENCIA** registra os problemas identificados nos laboratórios;
- Ela se relaciona com:
 - **USUARIO**: (através do campo **cpf_autor**) qualquer usuário pode registrar uma ocorrência.
 - **LABORATORIO**: local onde ocorreu o problema.
 - **CATEGORIA_OCORRENCIA**: classificação do tipo de problema.

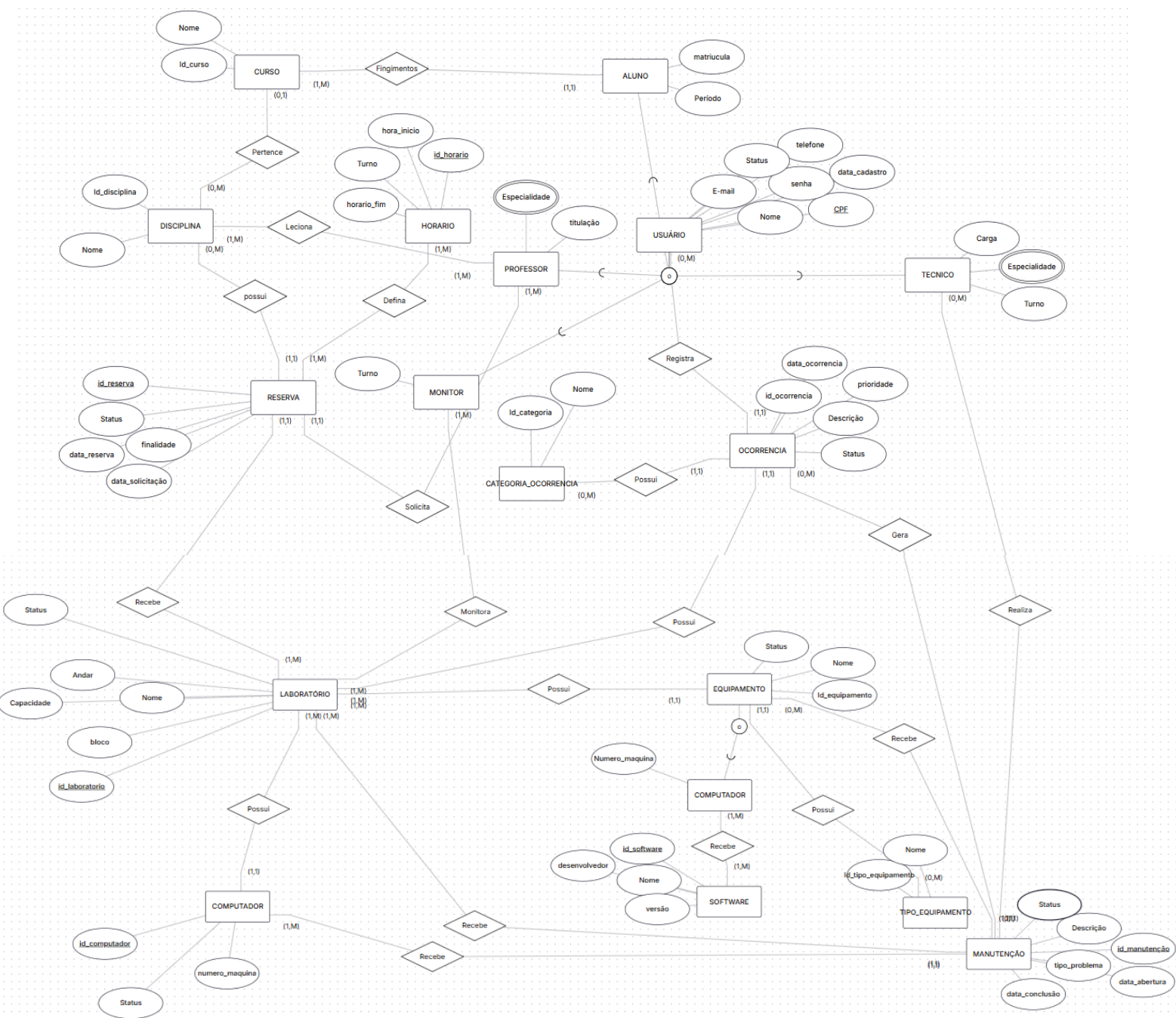
Fluxo de Manutenção

- A tabela **MANUTENCAO** registra as ações de reparo realizadas.
- Ela se relaciona com:
 - **OCORRENCIA** (pode ou não estar vinculada a uma ocorrência)
 - **EQUIPAMENTO** (equipamento que foi reparado)
- Permite registrar tanto manutenções corretivas quanto preventivas.

Associação de Softwares (N:M)

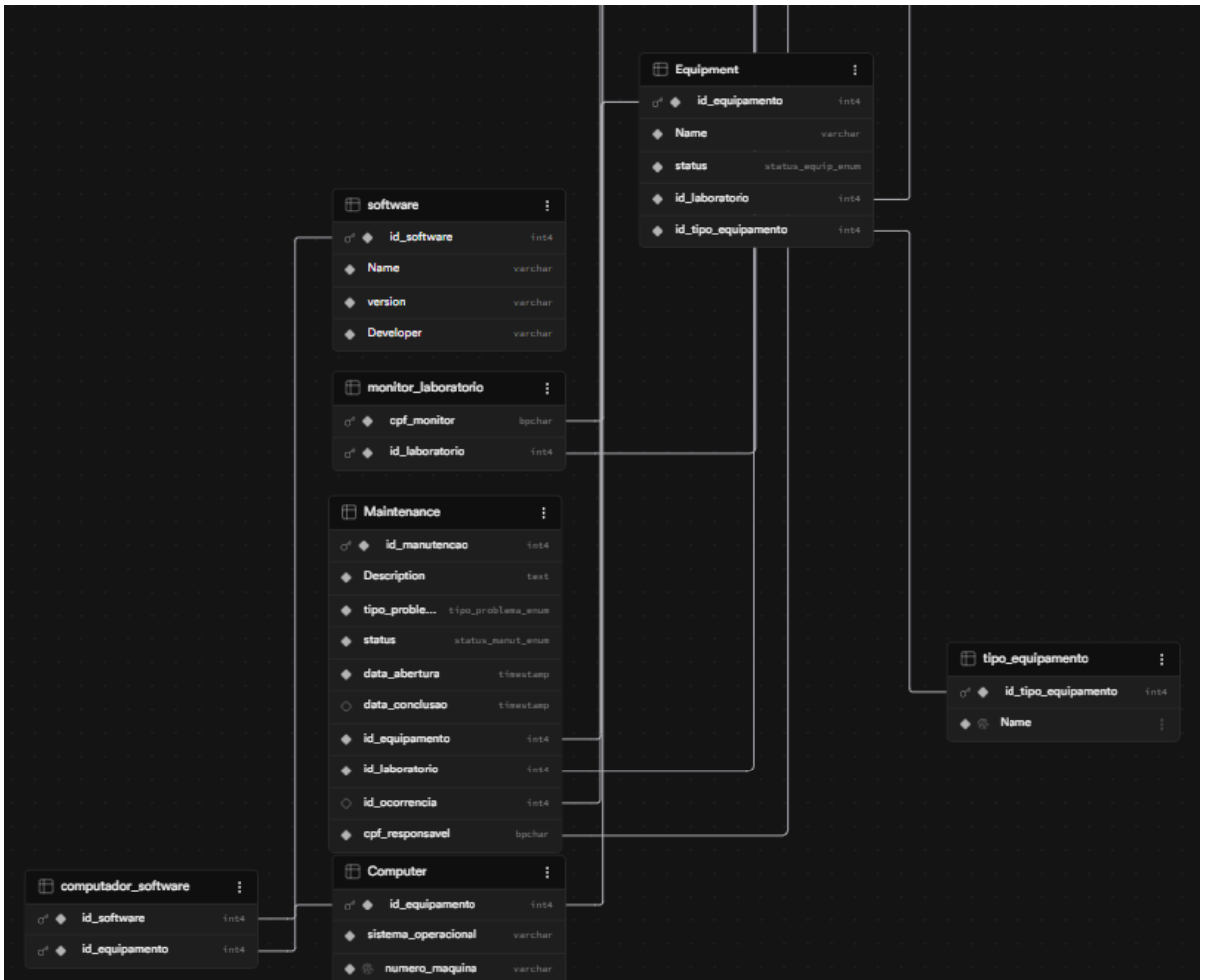
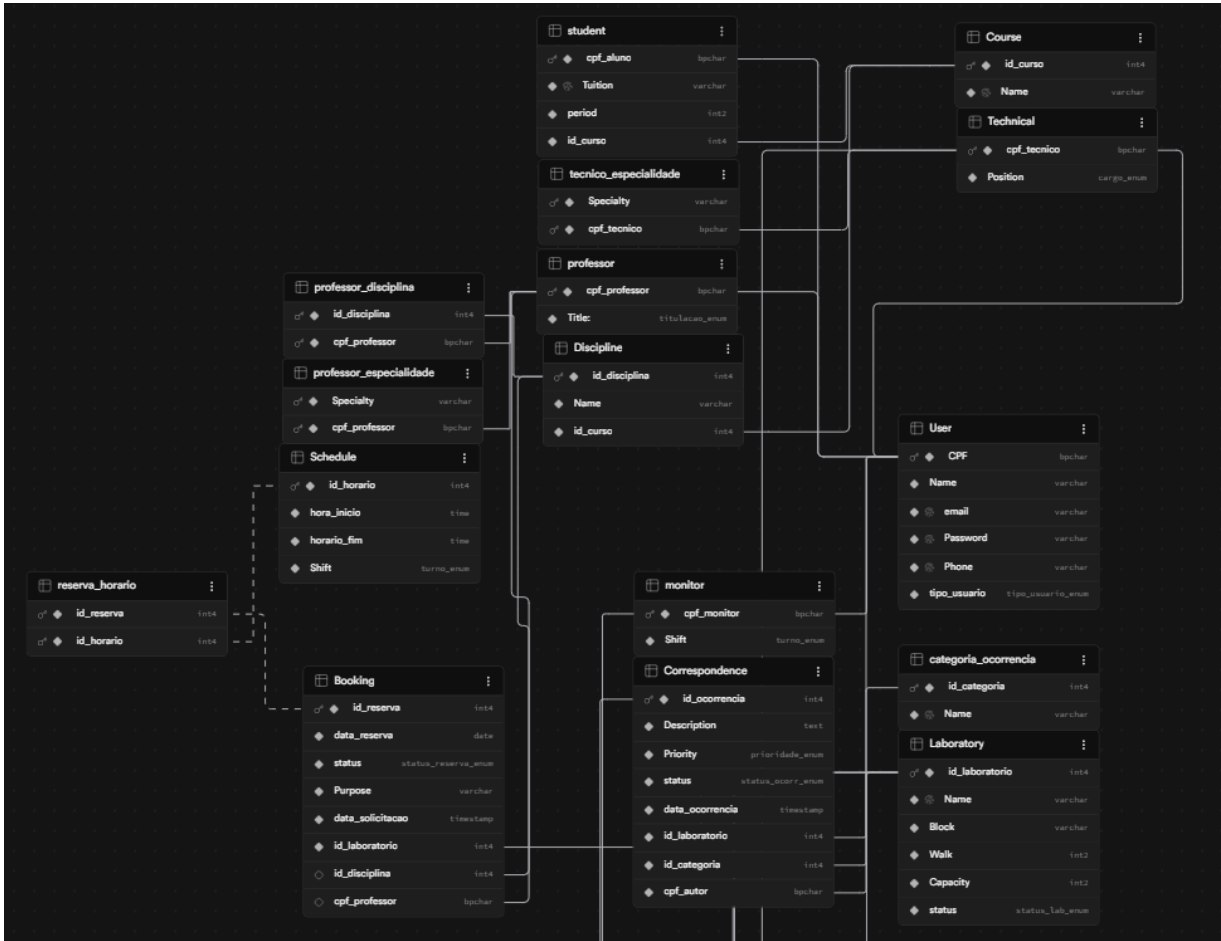
- A tabela **COMPUTADOR_SOFTWARE** é uma tabela associativa que implementa o relacionamento **N:M** entre **COMPUTADOR** e **SOFTWARE**.
- Um computador pode ter vários softwares instalados.
- Um mesmo software pode estar instalado em vários computadores.
- Essa relação não é feita diretamente com a tabela **EQUIPAMENTO**, garantindo que apenas computadores recebam softwares.

3.4. Modelo Entidade Relacionamento – (MER)



4. Modelo Lógico

4.1. Diagrama Entidade Relacionamento – (DER)



4.2. Criação de Tabelas

```
12
13
14 CREATE TABLE COURSE (
15     id_curso SERIAL PRIMARY KEY,
16     nome VARCHAR(100) NOT NULL UNIQUE
17 );
18
19
20 CREATE USER TABLE (
21     CPF CHAR(14) PRIMARY KEY,
22     nome VARCHAR(100) NOT NULL,
23     email VARCHAR(150) NOT NULL UNIQUE,
24     senha VARCHAR(255) NOT NULL UNIQUE,
25     telefone VARCHAR(20) NOT NULL UNIQUE,
26     tipo_usuario tipo_usuario_enum NOT NULL
27 );
28
29
```

```
29
30 CREATE TABLE ALUNO (
31     cpf_aluno CHAR(14) PRIMARY KEY REFERENCES USUARIO (CPF),
32     matricula VARCHAR(20) NOT NULL UNIQUE,
33     periodo SMALLINT NOT NULL,
34     id_curso INT NOT NULL REFERENCES CURSO (id_curso)
35 );
36
37
38 CREATE TABLE PROFESSOR (
39     cpf_professor CHAR(14) PRIMARY KEY REFERENCES USUARIO (CPF),
40     titulacao titulacao_enum NOT NULL
41 );
42
43
```



```
44 CREATE TABLE PROFESSOR_ESPECIALIDADE (  
45     specialty VARCHAR(100) NOT NULL,  
46     cpf_professor CHAR(14) NOT NULL REFERENCES PROFESSOR (cpf_professor),  
47     PRIMARY KEY (especialidade, cpf_professor)  
48 );  
49  
50  
51 CREATE TABLE TECNICO (  
52     cpf_tecnico CHAR(14) PRIMARY KEY REFERENCES USUARIO (CPF),  
53     cargo cargo_enum NOT NULL  
54 );  
55  
56  
57 CREATE TABLE TECNICO_ESPECIALIDADE (  
58     specialty VARCHAR(100) NOT NULL,  
59     cpf_tecnico CHAR(14) NOT NULL REFERENCES TECHNICAL (cpf_tecnico),  
60     PRIMARY KEY (especialidade, cpf_tecnico)  
61 );
```

```
64 CREATE TABLE MONITOR (  
65     cpf_monitor CHAR(14) PRIMARY KEY REFERENCES USUARIO (CPF),  
66     round turno_enum NOT NULL  
67 );  
68  
69  
70 CREATE TABLE DISCIPLINA (  
71     id_disciplina SERIAL PRIMARY KEY,  
72     nome VARCHAR(100) NOT NULL,  
73     id_curso INT NOT NULL REFERENCES CURSO (id_curso)  
74 );  
75  
76  
77 CREATE TABLE PROFESSOR_DISCIPLINA (  
78     id_disciplina INT NOT NULL REFERENCES DISCIPLINE (id_disciplina),  
79     cpf_professor CHAR(14) NOT NULL REFERENCES PROFESSOR (cpf_professor),  
80     PRIMARY KEY (id_disciplina, cpf_professor)  
81 );
```

```

82
83
84 CREATE TABLE HORARIO (
85     id_horario SERIAL PRIMARY KEY,
86     hora_inicio TIME NOT NULL,
87     horario_fim TIME NOT NULL,
88     round turno_enum NOT NULL
89 );
90
91
92 CREATE TABLE LABORATORIO (
93     id_laboratorio SERIAL PRIMARY KEY,
94     nome VARCHAR(100) NOT NULL UNIQUE,
95     bloco VARCHAR(20) NOT NULL,
96     andar SMALLINT NOT NULL,
97     capacidade SMALLINT NOT NULL,
98     status status_lab_enum NOT NULL DEFAULT 'ATIVO'
99 );

```

```

100
101
102 CREATE TABLE MONITOR_LABORATORIO (
103     cpf_monitor CHAR(14) NOT NULL REFERENCES MONITOR (cpf_monitor),
104     id_laboratorio INT NOT NULL REFERENCES LABORATORY (id_laboratorio),
105     PRIMARY KEY (cpf_monitor, id_laboratorio)
106 );
107
108
109 CREATE TABLE TIPO_EQUIPAMENTO (
110     id_tipo_equipamento SERIAL PRIMARY KEY,
111     nome VARCHAR(80) NOT NULL UNIQUE
112 );
113
114

```

```

115 CREATE TABLE EQUIPMENT (
116     id_equipamento SERIAL PRIMARY KEY,
117     nome VARCHAR(100) NOT NULL,
118     status status Equip_enum NOT NULL DEFAULT 'ATIVO',
119     id_laboratorio INT NOT NULL REFERENCES LABORATORY (id_laboratorio),
120     id_tipo_equipamento INT NOT NULL REFERENCES TIPO_EQUIPAMENTO (id_tipo_equipamento)
121 );
122
123
124 CREATE TABLE SOFTWARE (
125     id_software SERIAL PRIMARY KEY,
126     nome VARCHAR(100) NOT NULL,
127     versao VARCHAR(30) NOT NULL,
128     developer VARCHAR(100) NOT NULL,
129     UNIQUE (name, versao)
130 );
131

```

```
132
133 CREATE TABLE COMPUTER (
134   id Equipamento INT PRIMARY KEY REFERENCES EQUIPAMENTO (id Equipamento),
135   sistema_operacional VARCHAR(80) NOT NULL,
136   numero_maquina VARCHAR(30) NOT NULL UNIQUE
137 );
138
139
140 CREATE TABLE COMPUTADOR_SOFTWARE (
141   id Software INT NOT NULL REFERENCES SOFTWARE (id Software),
142   id Equipamento INT NOT NULL REFERENCES COMPUTADOR (id Equipamento),
143   PRIMARY KEY (id Software, id Equipamento)
144 );
145
146
```

```
147 CREATE TABLE RESERVA (
148   id Reserva SERIAL PRIMARY KEY,
149   data_Reserva DATE NOT NULL,
150   status status_Reserva_enum NOT NULL DEFAULT 'PENDING',
151   purpose VARCHAR(255) NOT NULL,
152   data_solicitacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
153   id Laboratorio INT NOT NULL REFERENCES LABORATORY (id Laboratorio),
154   id_disciplina INT REFERENCES DISCIPLINE (id_disciplina),
155   cpf_professor CHAR(14) REFERENCES PROFESSOR (cpf_professor)
156 );
157
158
159 CREATE TABLE RESERVA_HORARIO (
160   id Reserva INT NOT NULL REFERENCES RESERVE (id Reserva),
161   id_horario INT NOT NULL REFERENCES HORARIO (id_horario),
162   PRIMARY KEY (id Reserva, id_horario)
163 );
```

```

165
166 CREATE TABLE CATEGORIA_OCORRENCIA (
167     id_categoria SERIAL PRIMARY KEY,
168     nome VARCHAR(80) NOT NULL UNIQUE
169 );
170
171
172 CREATE TABLE OCORRENCIA (
173     id_ocorrencia SERIAL PRIMARY KEY,
174     descricao TEXT NOT NULL,
175     prioridade prioridade_enum NOT NULL DEFAULT 'MEDIA',
176     status status_ocorr_enum NOT NULL DEFAULT 'ABERTA',
177     data_ocorrencia TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
178     id_laboratorio INT NOT NULL REFERENCES LABORATORY (id_laboratorio),
179     id_categoria INT NOT NULL REFERENCES CATEGORIA_OCORRENCIA (id_categoria),
180     cpf_autor CHAR(14) NOT NULL REFERENCES USUARIO (CPF)
181 );
182

```

```

183
184 CREATE TABLE MANUTENCAO (
185     id_manutencao SERIAL PRIMARY KEY,
186     descricao TEXT NOT NULL,
187     tipo_problema tipo_problema_enum NOT NULL,
188     status status_manut_enum NOT NULL DEFAULT 'ABERTA',
189     data_abertura TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
190     data_conclusao TIMESTAMP,
191     id_equipamento INT NOT NULL REFERENCES EQUIPAMENTO (id_equipamento),
192     id_laboratorio INT NOT NULL REFERENCES LABORATORY (id_laboratorio),
193     id_ocorrencia INT REFERENCES OCORRENCIA (id_ocorrencia),
194     cpf_responsavel CHAR(14) NOT NULL REFERENCES TECHNICAL (cpf_tecnico)
195 );
196
197

```

5. Modelo Físico

5.1. Povoamento

Tabela Usuário

INSERT INTO USUARIO (CPF, nome, email, senha, telefone, tipo_usuario) VALUES

('167.826.000-64', 'Andrisson Heitor', 'andrisson5533@gmail.com', 'Heitor0101@', '(84) 997877634', 'ALUNO'),

13

('180.800.000-12', 'Lucas Silva', 'lucassilva555@gmail.com', 'Lucas555silva', '(84) 991208080', 'PROFESSOR'),
('120.001.090-34', 'Maria Beatriz', 'beatriz005@gmail.com', 'beatriz005@', '(84) 99602-0000', 'MONITOR'),
('700.800.900-01', 'Ricardo Sousa', 'ricardo000@gmail.com', 'sousa0808@', '(84) 99000-5555', 'TECNICO'),
('100.200.300-22', 'Emanoel Melo', 'melo000@gmail.com', 'melo0000@', '(84) 99000-5508', 'PROFESSOR'),
('900.456.789-99', 'Luiz Carlos', 'luiz@email.com', '000@luiz', '(84) 00000-9999', 'ALUNO'),
('111.222.333-44', 'Aísha Marinho', 'marinho.tecnico@uni.edu', '246810@', '(81) 981546328', 'TECNICO'),
('555.666.777-88', 'Maria José', 'maria.tecnica@uni.edu', '13579@', '(81) 991863425', 'TECNICO'),
('999.888.777-66', 'Jailson Calado', 'jailson.tecnico@uni.edu', '36912@', '(87) 981756324', 'TECNICO'),
('987.654.321-01', 'Rhuan Almeida', 'rhuan1@gmail.com', 'Rhuan@23', '(87) 991564872', 'PROFESSOR'),
('987.654.321-02', 'Kauã Reclione', 'kauare@gmail.com', 'Kauã17@', '(87) 991576252', 'PROFESSOR'),
('586.496.754-87', 'Jeffeson Sousa', 'jerfferson3n@gmail.com', 'Jeff@769', '(87) 99999-9999', 'ALUNO'),
('579.123.456-85', 'Jorge de Melo', 'jorge07@gmail.com', 'Jorge32@90', '(81) 99256-7881', 'ALUNO'),
('635.478.521-89', 'Maria Helena', 'helenal4@gmail.com', 'heLena1417@', '(87) 99264-1665', 'ALUNO'),
('456.821.564-78', 'Levi Ackerman', 'levizin31@gmail.com', 'MTitanM@', '(87) 9918-5882', 'ALUNO'),
('156.256.479-98', 'Edward Elric', 'elric67@gmail.com', 'Alquimia@66', '(87) 98256-6545', 'ALUNO'),
('123.987.456-02', 'Keila Cristina', 'keila01@gmail.com', 'keilinha@02', '(81) 98197-7975', 'ALUNO');

Tabela Aluno

```
INSERT INTO ALUNO (cpf_aluno, matricula, periodo, id_curso) VALUES  
(167.826.000-64, 1400900, 3, 1),  
(635.478.521-89, 0022040, 3, 1),  
(123.987.456-02, 1099199, 3, 1),  
(456.821.564-78, 8008880, 3, 1),  
(586.496.754-87, 7077077, 5, 2),  
(579.123.456-85, 6066600, 5, 2),
```

('156.256.479-98', '1102110', 5, 2);

14

Tabela Professor

```
INSERT INTO PROFESSOR (cpf_professor, titulacao) VALUES  
(180.800.000-12, 'MESTRE'),  
(100.200.300-22, 'DOUTOR'),  
(987.654.321-01, 'DOUTOR'),  
(987.654.321-02, 'DOUTOR');
```

Tabela Professor Disciplina

```
INSERT INTO professor_disciplina (id_disciplina, cpf_professor) VALUES  
(1, 222.222.222-22),  
(2, 100.200.300-22),  
(3, 222.222.222-22),  
(4, 100.200.300-22),  
(5, 987.654.321-01),  
(6, 987.654.321-02);
```

Tabela Professor Especialidade

```
INSERT INTO PROFESSOR_ESPECIALIDADE (especialidade, cpf_professor) VALUES  
(Inteligencia Artificial, 180.800.000-12),  
(Estruturas de Dados, 100.200.300-22),  
(Programação em Python, 987.654.321-01),  
(Programação em Java, 987.654.321-02),  
(Banco de Dados, 222.222.222-22),  
(Engenharia de Software, 222.222.222-22);
```

Tabela Monitor

```
INSERT INTO MONITOR (cpf_monitor, turno) VALUES  
(120.001.090-34, TARDE),  
(333.333.333-33, MANHA);
```

Tabela Monitor Laboratório

```
INSERT INTO MONITOR_LABORATORIO (cpf_monitor, id_laboratorio) VALUES  
(120.001.090-34, 2),  
(333.333.333-33, 1);
```

Tabela Técnico

```
INSERT INTO TECNICO (cpf_tecnico, cargo) VALUES  
(700.800.900-01, TECNICO_ELETRICO),  
(111.222.333-44, TECNICO_TI),
```

('555.666.777-88', 'TECNICO_TI'),
('999.888.777-66', 'TECNICO_ELETRICO'),

15

('444.444.444-44', 'TECNICO_TI');

Tabela Técnico Especialidade

```
INSERT INTO TECNICO_ESPECIALIDADE (especialidade, cpf_tecnico) VALUES  
( 'Manutenção Elétrica', '700.800.900-01'),  
( 'Manutenção de Software', '444.444.444-44'),  
( 'Segurança da Informação', '700.800.900-01'),  
( 'Manutencao de Hardware', '444.444.444-44'),  
( 'Redes e Infraestrutura', '444.444.444-44');
```

Tabela Curso

```
INSERT INTO CURSO (nome) VALUES  
( 'licenciatura_em_Computação'),  
( 'Engenharia_de_Software');
```

Tabela Disciplina

```
INSERT INTO disciplina (id_disciplina, nome, id_curso) VALUES  
(1, 'Banco de Dados', 1),  
(2, 'Algoritmos e Estruturas', 1),  
(3, 'Engenharia de Requisitos', 2),  
(4, 'Qualidade de Software', 2),  
(5, 'Programação 1', 1),  
(6, 'Programação 2', 2);
```

Tabela Laboratório

```
INSERT INTO LABORATORIO (nome, bloco, andar, capacidade, status) VALUES  
( 'Lab de Computação A', 'Bloco A', 1, 30, 'ATIVO'),  
( 'Lab de Computação B', 'Bloco A', 2, 25, 'ATIVO'),  
( 'Lab de Redes', 'Bloco B', 1, 20, 'EM_MANUTENCAO');
```

Tabela Equipamento

```
INSERT INTO EQUIPAMENTO (id_equipamento, nome, status, id_laboratorio,  
id_tipo_equipamento) VALUES  
(1, 'PC-01', 'ATIVO', 1, 1),  
(2, 'PC-02', 'ATIVO', 1, 1),  
(3, 'PC-03', 'ATIVO', 1, 1),  
(4, 'PC-04', 'EM_MANUTENCAO', 1, 1),  
(5, 'PC-05', 'ATIVO', 1, 1),  
(6, 'Projektor Lab A', 'ATIVO', 1, 2),
```

(7, 'Switch Lab A', 'ATIVO', 1, 3),
(8, 'PC-06', 'ATIVO', 2, 1),

16

(9, 'PC-07', 'ATIVO', 2, 1),
(10, 'PC-08', 'ATIVO', 2, 1),
(11, 'Projektor Lab B', 'ATIVO', 2, 2),
(12, 'Switch Lab B', 'ATIVO', 2, 3);

Tabela Tipo de Equipamento

```
INSERT INTO TIPO_EQUIPAMENTO (nome) VALUES  
( 'Computador Desktop'),  
( 'Projektor'),  
( 'Switch de Rede'),  
( 'Impressora');
```

Tabela Computador

```
INSERT INTO COMPUTADOR (id_equipamento, sistema_operacional, numero_maquina)  
VALUES  
(1, 'Ubuntu 22.04', 'LAB-A-001'),  
(2, 'Ubuntu 22.04', 'LAB-A-002'),  
(3, 'Windows 11', 'LAB-A-003'),  
(4, 'Windows 11', 'LAB-A-004'),  
(5, 'Ubuntu 22.04', 'LAB-A-005'),  
(6, 'Windows 11', 'LAB-A-006'),  
(7, 'Ubuntu 22.04', 'LAB-B-005'),  
(8, 'Windows 11', 'Lab-B-006'),  
(9, 'Windows 11', 'Lab-B-007'),  
(10, 'Windows 11', 'Lab-B-008'),
```

Tabela Computador Software

```
INSERT INTO COMPUTADOR_SOFTWARE (id_software, id_equipamento) VALUES  
(1, 1), (2, 1), (3, 1), (4, 1), (1, 2), (2, 2), (4, 2), (1, 3), (4, 3), (5, 3), (4, 4), (1, 5), (2, 5), (3, 5);
```

Tabela Software

```
INSERT INTO SOFTWARE (nome, versao, desenvolvedor) VALUES  
( 'VS Code', '1.89', 'Microsoft'),  
( 'Python', '3.12', 'Python Foundation'),  
( 'PostgreSQL', '16.0', 'PostgreSQL Global'),  
( 'Navegador Firefox', '126.0', 'Mozilla'),  
( 'IntelliJ IDEA', '2024.1', 'JetBrains');
```

Tabela Reserva


```
INSERT INTO RESERVA (id_reserva, data_reserva, status, finalidade, id_laboratorio, id_disciplina, cpf_professor) VALUES
```

17

```
('2025-07-10', 'APROVADA', 'Aula pratica de SQL e consultas em banco de dados.', 1, 1, '222.222.222-22'),  
(2, '2026-06-20', 'APROVADA', 'Aula pratica de Pilhas e Filas', 2, 2, '100.200.300-22'),  
(3, '2026-07-15', 'APROVADA', 'Prova de Banco de Dados.', '2026-06-05 21:32', 2, 1, '222.222.222-22'),  
(4, '2026-07-20', 'APROVADA', 'Introdução ao Python "hello word".', '2026-06-09 17:29', 1, 5, '987.654.321-01'),  
(5, '2026-07-20', 'APROVADA', 'Introdução a Programação Orientada a Objetos.', '2026-06-18 15:16', 2, 6, '987.654.321-02');
```

Tabela Horário

```
INSERT INTO HORARIO (hora_inicio, horario_fim, turno) VALUES  
( '07:00', '08:40', 'MANHA'),  
( '08:50', '10:30', 'MANHA'),  
( '13:00', '14:40', 'TARDE'),  
( '14:50', '16:30', 'TARDE'),  
( '19:00', '20:40', 'NOITE'),  
( '20:50', '22:30', 'NOITE');
```

Tabela Reserva Horário

```
INSERT INTO reserva_horario (id_reserva, id_horario) VALUES  
(1, 1),  
(1, 2),  
(3, 3),  
(4, 3),  
(5, 5);
```

Tabela Ocorrência

```
INSERT INTO OCORRENCIA (descricao, prioridade, status, id_laboratorio, id_categoria, cpf_autor) VALUES  
( 'PC-04 nao liga, possivelmente problema na fonte de alimentacao.', 'ALTA', 'ABERTA', 1, 1, '111.111.111-11'),  
( 'PC-03 com problema no monitor', 'ALTA', 'ABERTA', 2, 1, '167.826.000-64'),  
( 'PC-08 com problema na saida de som', 'ALTA', 'ABERTA', 2, 1, '120.001.090-34'),  
( 'Projeto Lab B nao funciona', 'ALTA', 'ABERTA', 2, 1, '180.800.000-12'),  
( 'PC-01, VS Code esta desatualizado', 'MEDIA', 'ABERTA', 1, 2, '167.826.000-64'),
```

Tabela Categoria Ocorrência

```
INSERT INTO CATEGORIA_OCORRENCIA (nome) VALUES  
(('Equipamento com Defeito'),
```

18

```
('Problema de Software'),  
(('Problema de Rede'),  
(('Infraestrutura');
```

Tabela Manutenção

```
INSERT INTO MANUTENCAO (descricao, tipo_problema, status, id_equipamento,  
id_laboratorio, id_ocorrencia, cpf_responsavel) VALUES  
(('Verificar e consertar Projetor do lab B.', 'HARDWARE', 'ABERTA', 11, 2, 4,  
'700.800.900-01'),  
(('Verificar e consertar problema com monitor do PC-03.', 'HARDWARE', 'ABERTA', 3, 1, 2,  
'700.800.900-01'),  
(('Verificar e substituir fonte de alimentacao do PC-04.', 'HARDWARE', 'ABERTA', 4, 1, 1,  
'444.444.444-44');
```

6. Principais Views

View 1: Laboratórios

```
CREATE VIEW vw_laboratorios_disponiveis AS  
SELECT  
id_laboratorio,  
nome,  
bloco,  
andar,  
capacidade  
FROM laboratorio;
```

View 2: Reservas detalhadas

```
CREATE VIEW vw_reservas_detalhadas AS  
SELECT  
r.id_reserva,  
r.data_reserva,  
r.finalidade,  
r.status,  
l.nome AS laboratorio  
FROM reserva r  
JOIN laboratorio l  
ON r.id_laboratorio = l.id_laboratorio;
```

View 3: Professor e disciplina

```
CREATE VIEW vw_professor_disciplina AS
SELECT
pd.cpf_professor,
d.nome AS disciplina
FROM professor_disciplina pd
JOIN disciplina d
ON pd.id_disciplina = d.id_disciplina;
```

View 4: Reservas por laboratório

```
CREATE VIEW vw_reservas_laboratorio AS
SELECT
l.nome AS laboratorio,
r.id_reserva,
r.data_reserva,
r.status
FROM laboratorio l
JOIN reserva r
ON l.id_laboratorio = r.id_laboratorio;
```

View 5: Quantidade de reservas por laboratório

```
CREATE VIEW vw_qtd_reservas_laboratorio AS
SELECT
l.nome,
COUNT(r.id_reserva) AS total_reservas
FROM laboratorio l
LEFT JOIN reserva r
ON l.id_laboratorio = r.id_laboratorio
GROUP BY l.nome;
```

View 6: Status dos laboratorios

```
CREATE OR REPLACE VIEW vw_laboratorios_status AS
SELECT
l.nome,
l.status,
```

```

COUNT(DISTINCT e.id_equipamento) AS total_equipamentos,

COUNT(DISTINCT CASE
  WHEN e.status = 'ATIVO'
  THEN e.id_equipamento
END) AS equipamentos_ativos,

COUNT(DISTINCT CASE
  WHEN e.status = 'EM_MANUTENCAO'
  THEN e.id_equipamento
END) AS equipamentos_em_manutencao,

COUNT(DISTINCT c.id_equipamento) AS total_computadores

FROM laboratorio l
LEFT JOIN equipamento e
  ON e.id_laboratorio = l.id_laboratorio
LEFT JOIN computador c
  ON c.id_equipamento = e.id_equipamento

GROUP BY
  l.id_laboratorio,
  l.nome,
  l.status;

```

7: Visualizar ocorrencia

```

CREATE OR REPLACE VIEW vw_ocorrencias_abertas AS
SELECT
  o.id_ocorrencia,
  l.nome AS laboratorio,
  c.nome AS categoria,
  o.prioridade,
  o.status,
  u.nome AS autor
FROM ocorrencia o
JOIN laboratorio l

```

```

    ON l.id_laboratorio = o.id_laboratorio
JOIN categoria ocorrência c
    ON c.id_categoria = o.id_categoria
JOIN usuario u
    ON u.cpf = o.cpf autor
WHERE o.status = 'ABERTA';

```

7. Principais Funções

1: Verifica se um laboratório está disponível e um horário específico

```

CREATE OR REPLACE FUNCTION fn_laboratorio_disponivel(
  p_id_laboratorio INT,
  p_data          DATE,
  p_id_horario    INT
)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
DECLARE
  v_conflito INT;
BEGIN
  SELECT COUNT(*)
  INTO v_conflito
  FROM RESERVA r
  JOIN RESERVA_HORARIO rh ON r.id_reserva = rh.id_reserva
  WHERE r.id_laboratorio = p_id_laboratorio
    AND r.data_reserva = p_data
    AND rh.id_horario = p_id_horario
    AND r.status IN ('PENDENTE', 'APROVADA');

  RETURN v_conflito = 0;
END;
$$;

```

Como chamar a função: SELECT fn_laboratorio_disponivel (2, 26-08-12, 2);

2: Retorna o total de reservas de um professor e um intervalo de tempo

```

CREATE OR REPLACE FUNCTION fn_reservas_professor(
  p_cpf_professor CHAR(14),
  p_data_inicio  DATE,
  p_data_fim     DATE
)
RETURNS TABLE (
  id_reserva INT,
  data_reserva DATE,

```

```

status      status_reserva_enum,
laboratorio VARCHAR,
finalidade  VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        r.id_reserva,
        r.data_reserva,
        r.status,
        l.nome::VARCHAR AS laboratorio,
        r.finalidade::VARCHAR
    FROM RESERVA r
    JOIN LABORATORIO l ON r.id_laboratorio = l.id_laboratorio
    WHERE r.cpf_professor = p_cpf_professor
        AND r.data_reserva BETWEEN p_data_inicio AND p_data_fim
    ORDER BY r.data_reserva;
END;
$$;

```

Como chamar a função: SELECT * FROM fn_reservas_professor('222.222.222-22', '2025-01-01', '2025-12-31');

3: Retorna um resumo de ocorrências

```

CREATE OR REPLACE FUNCTION fn_resumo_ocorrencias(
    p_id_laboratorio INT
)
RETURNS TABLE (
    status      status_ocorr_enum,
    total       BIGINT,
    mais_recente TIMESTAMP
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM LABORATORIO WHERE id_laboratorio = p_id_laboratorio
    ) THEN
        RAISE EXCEPTION 'Laboratório % não encontrado.', p_id_laboratorio;
    END IF;

    RETURN QUERY

```

```

SELECT
    o.status,
    COUNT(*)      AS total,
    MAX(o.data_ocorrencia) AS mais_recente
FROM OCORRENCIA o
WHERE o.id_laboratorio = p_id_laboratorio
GROUP BY o.status
ORDER BY total DESC;
END;
$$;

```

Como chamar a função: SELECT * FROM fn_resumo_ocorrencias(1);

4: Troca o tipo de usuário

```

CREATE OR REPLACE FUNCTION fn_trocar_tipo_usuario(
    p_cpf      CHAR(14),
    p_novo_tipo tipo_usuario_enum
)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_tipo_atual tipo_usuario_enum;
BEGIN
    SELECT tipo_usuario INTO v_tipo_atual
    FROM USUARIO
    WHERE CPF = p_cpf;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Usuário % não encontrado.', p_cpf;
    END IF;

    IF v_tipo_atual = p_novo_tipo THEN
        RAISE EXCEPTION 'Usuário já é %.', p_novo_tipo;
    END IF;

    DELETE FROM ALUNO   WHERE cpf_aluno   = p_cpf;
    DELETE FROM PROFESSOR WHERE cpf_professor = p_cpf;
    DELETE FROM MONITOR  WHERE cpf_monitor  = p_cpf;
    DELETE FROM TECNICO  WHERE cpf_tecnico  = p_cpf;

    UPDATE USUARIO
    SET tipo_usuario = p_novo_tipo
    WHERE CPF = p_cpf;

```

```

RETURN 'Tipo alterado de ' || v_tipo_atual || ' para ' || p_novo_tipo;
END;
$$;

```

Como chamar a função: SELECT fn_trocar_tipo_usuario('123.456.789-00', 'PROFESSOR');

5: Relatório geral

```

CREATE OR REPLACE FUNCTION fn_relatorio_geral()
RETURNS TABLE (
    total_usuarios    BIGINT,
    total_alunos      BIGINT,
    total_professores BIGINT,
    total_monitores   BIGINT,
    total_tecnicos    BIGINT,
    total_laboratorios BIGINT,
    labs_ativos       BIGINT,
    labs_manutencao   BIGINT,
    total_equipamentos BIGINT,
    equip_ativos      BIGINT,
    equip_manutencao  BIGINT,
    reservas_pendentes BIGINT,
    reservas_aprovadas BIGINT,
    ocorrencias_abertas BIGINT,
    manutencoes_abertas BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        (SELECT COUNT(*) FROM USUARIO)                AS total_usuarios,
        (SELECT COUNT(*) FROM ALUNO)                    AS total_alunos,
        (SELECT COUNT(*) FROM PROFESSOR)                AS total_professores,
        (SELECT COUNT(*) FROM MONITOR)                  AS total_monitores,
        (SELECT COUNT(*) FROM TECNICO)                   AS total_tecnicos,
        (SELECT COUNT(*) FROM LABORATORIO)               AS total_laboratorios,
        (SELECT COUNT(*) FROM LABORATORIO WHERE status = 'ATIVO') AS labs_ativos,
        (SELECT COUNT(*) FROM LABORATORIO WHERE status = 'EM_MANUTENCAO') AS
labs_manutencao,
        (SELECT COUNT(*) FROM EQUIPAMENTO)               AS total_equipamentos,
        (SELECT COUNT(*) FROM EQUIPAMENTO WHERE status = 'ATIVO') AS equip_ativos,
        (SELECT COUNT(*) FROM EQUIPAMENTO WHERE status = 'EM_MANUTENCAO') AS
equip_manutencao,

```



```

        (SELECT COUNT(*) FROM RESERVA WHERE status = 'PENDENTE')          AS
reservas_pendentes,
        (SELECT COUNT(*) FROM RESERVA WHERE status = 'APROVADA')          AS
reservas_aprovadas,
        (SELECT COUNT(*) FROM OCORRENCIA WHERE status IN ('ABERTA','EM_ANDAMENTO')) AS
ocorrencias_abertas,
        (SELECT COUNT(*) FROM MANUTENCAO WHERE status IN ('ABERTA','EM_ANDAMENTO')) AS
manutencoes_abertas;
END;
$$;

```

Como chamar a função: SELECT * FROM fn_relatorio_geral();

8. Principais Triggers

1: Impede reserva em laboratório em manutenção ou inativo

```

CREATE OR REPLACE FUNCTION trg_fn_validar_reserva()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_status_lab status_lab_enum;
BEGIN
    SELECT status INTO v_status_lab
    FROM LABORATORIO
    WHERE id_laboratorio = NEW.id_laboratorio;
    IF v_status_lab <> 'ATIVO' THEN
        RAISE EXCEPTION
            'Não é possível reservar o laboratório %. Status atual: %',
            NEW.id_laboratorio, v_status_lab;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validar_reserva
BEFORE INSERT ON RESERVA
FOR EACH ROW
EXECUTE FUNCTION trg_fn_validar_reserva();

```

2: Toda vez que uma manutenção for concluída, reativa o equipamento automaticamente

```

CREATE OR REPLACE FUNCTION trg_fn_concluir_manutencao()

```

```

RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status = 'CONCLUIDA' AND OLD.status <> 'CONCLUIDA' THEN
        -- Registra data de conclusão se não informada
        IF NEW.data_conclusao IS NULL THEN
            NEW.data_conclusao := CURRENT_TIMESTAMP;
        END IF;
        -- Reativa o equipamento
        UPDATE EQUIPAMENTO
        SET status = 'ATIVO'
        WHERE id_equipamento = NEW.id_equipamento;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_concluir_manutencao
BEFORE UPDATE ON MANUTENCAO
FOR EACH ROW
EXECUTE FUNCTION trg_fn_concluir_manutencao();

```

3: Validação de usuário

```

CREATE OR REPLACE FUNCTION trg_fn_validar_usuario()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.CPF !~ '^d{3}\.d{3}\.d{3}-d{2}$' THEN
        RAISE EXCEPTION 'CPF % inválido. Use o formato 000.000.000-00.', NEW.CPF;
    END IF;

    IF NEW.email !~ '^[^@]+@[^@]+\.[^@]+$' THEN
        RAISE EXCEPTION 'Email % inválido.', NEW.email;
    END IF;

    IF NEW.telefone !~ '^(?d{2})?s?d{4,5}-?d{4}$' THEN
        RAISE EXCEPTION 'Telefone % inválido.', NEW.telefone;
    END IF;

    RETURN NEW;
END;
$$;

```

```
CREATE TRIGGER trg_validar_usuario
BEFORE INSERT OR UPDATE ON USUARIO
FOR EACH ROW
EXECUTE FUNCTION trg_fn_validar_usuario();
```

4: Bloqueia conflito de horário em reservas

```
CREATE OR REPLACE FUNCTION trg_fn_conflito_horario()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_conflito INT;
BEGIN
    SELECT COUNT(*) INTO v_conflito
    FROM RESERVA_HORARIO rh
    JOIN RESERVA r ON rh.id_reserva = r.id_reserva
    WHERE r.id_laboratorio = NEW.id_laboratorio
    AND r.data_reserva = NEW.data_reserva
    AND rh.id_horario = NEW.id_horario
    AND r.status IN ('PENDENTE', 'APROVADA')
    AND r.id_reserva <> NEW.id_reserva;

    IF v_conflito > 0 THEN
        RAISE EXCEPTION 'Conflito de horário: laboratório % já reservado nesse horário.', NEW.id_laboratorio;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_conflito_horario
BEFORE INSERT ON RESERVA_HORARIO
FOR EACH ROW
EXECUTE FUNCTION trg_fn_conflito_horario();
```

9. Considerações Finais

Ao longo do desenvolvimento deste projeto de banco de dados para o **GestLab TI (Sistema de Agendamento de Laboratório de Informática)**, foram abordados diversos aspectos essenciais relacionados à modelagem, estruturação e implementação do sistema proposto. O objetivo principal foi criar um banco de dados eficiente e seguro que atenda às

demandas da reitoria, coordenação e dos docentes da instituição, garantindo a integridade dos dados e otimizando o controle de recursos, reservas e manutenções preventivas e corretivas.

Durante a fase de design conceitual, o uso da ferramenta ERDPlus foi de extrema importância para mapear com clareza o mini-mundo e suas regras. Posteriormente, no processo de transição para o modelo relacional e implementação física via VS Code, foram utilizados os princípios de normalização de dados para organizar as tabelas, minimizar redundâncias e eliminar dependências indesejadas. A integridade referencial foi estritamente assegurada por meio da definição correta de chaves primárias e estrangeiras aplicadas pela plataforma Supabase.

Como pontos positivos do projeto, destaca-se o sucesso no controle rígido de usuários, no catálogo detalhado de ativos de hardware/software e no fluxo automatizado que une solicitações de reservas a ordens de serviço para a equipe técnica.

A documentação gerada e o conjunto de scripts SQL desenvolvidos servem como um guia técnico robusto para o entendimento da estrutura do banco, suas relações e a lógica de armazenamento. Em conclusão, este projeto proporcionou uma experiência prática valiosa na disciplina de Banco de Dados do curso de Licenciatura em Computação da Universidade de Pernambuco (UPE), apontando a importância do alinhamento entre a engenharia de requisitos e as boas práticas de modelagem SQL para entregar uma solução eficaz, escalável e segura para a infraestrutura tecnológica educacional.